

# Module 1 : Programming in Python

Source : Python for Education, Ajith Kumar B.P., Jun 2010 <https://www.iuac.res.in/phoenix/python4schools/Python-for-Education.pdf> (<https://www.iuac.res.in/phoenix/python4schools/Python-for-Education.pdf>)

Python is a simple, high level language with a clean syntax. It offers strong support for integration with other languages and tools, comes with extensive standard libraries, and can be learned in a few days. Many Python programmers report substantial productivity gains and feel the language encourages the development of higher quality, more maintainable code. To know more visit the Python website.

<http://www.python.org/> (<http://www.python.org/>) and <https://docs.python.org/3/tutorial/> (<https://docs.python.org/3/tutorial/>)

## 1.1 Getting Started with Python

To start programming in Python, we have to learn how to type the source code and save it to a file, using a text editor program. We also need to know how to open a Command Terminal and start the Python Interpreter. The details of this process may vary from one system to another. On an Ubuntu GNU/Linux system, you can open the Text Editor and the Terminal from the Applications->Accessories menu.

### 1.1.1 Two modes of using Python Interpreter

If you issue the command 'python', without any argument, from the command terminal, the Python interpreter will start and display a '>>>' prompt where you can type Python statements. Use this method only for viewing the results of single Python statements, for example to use Python as a calculator. It could be confusing when you start writing larger programs, having looping and conditional statements. The preferred way is to enter your source code in a text editor, save it to a file (with .py extension) and execute it from the command terminal using Python. In this document, we will start writing small programs showing the essential elements of the language without going into the details. The reader is expected to run these example programs and also to modify them in different ways. It is like learning to drive a car, you master it by practicing. Let us start with a program to display the words Hello World on the computer screen. This is the customary 'hello world' program. There is another version that prints 'Goodbye cruel world', probably invented by those who give up at this point. The Python 'hello world' program is shown below.

Example. hello.py

```
In [1]: print('Hello World') #print 'hello world' won't work in Python3  
Hello World
```

This should be entered into a text file using any text editor. On a GNU/Linux system you may use the text editor like 'gedit' to create the source file, save it as hello.py. The next step is to call the Python Interpreter to execute the new program. For that, open a command terminal and (at the prompt) type:

```
python hello.py
```

There are a few changes in Python Programming over past few years. In 2008, Python3 came with a few major changes. One of the notable changes is the introduction of `**print()` function. `print` changed from being a built-in statement to a built-in function. Other changes include division operator(`/`) turned into true division, `range` turned into another type, `raw_input()` was depreciated and included unicode characters in strings.

## 1.2 Variables and Data Types

As mentioned earlier, any high level programming language should support several data types. The problem to be solved is represented using variables belonging to the supported data types. Python supports numeric data types like integers, floating point numbers and complex numbers. To handle character strings, it uses the String data type. Python also supports other compound data types like lists, tuples, dictionaries etc.

In languages like C, C++ and Java, we need to explicitly declare the type of a variable. This is not required in Python. The data type of a variable is decided by the value assigned to it. This is called dynamic data typing. The type of a particular variable can change during the execution of the program. If required, one type of variable can be converted in to another type by explicit type casting, like `y = float(3)`. Strings are enclosed within single quotes or double quotes.

The program *first.py* shows how to define variables of different data types. It also shows how to embed comments inside a program. The output of the program is shown below. Note that the type of the variable `x` changes during the execution of the program, depending on the value assigned to it.

```
In [2]: '''
        A multi-line comment, within a pair of three single quotes.
        In a line, anythin after a # sign is also a comment.
        '''
        x = 10
        print (x,type(x))
        x = 10.4
        print(x,type(x))
        x = 3+4j
        print(x,type(x))
        x = 'I am a String'
        print(x,type(x))

10 <class 'int'>
10.4 <class 'float'>
(3+4j) <class 'complex'>
I am a String <class 'str'>
```

The program treats the variables like humans treat labelled envelopes. We can pick an envelope, write some name on it and keep something inside it for future use. In a similar manner the program creates a variable, gives it a name and keeps some value inside it, to be used in subsequent steps. So far we have used four data types of Python: int, float, complex and str. To become familiar with them, you may write simple programs performing arithmetic and logical operations using them. Running the program *oper.py* will generate the following output.

```
In [3]: x = 2
        y = 4
        print(x+y*2)
        s = 'Hello '
        print(s+s)
        print(3*s)
        print(x==y)
        print(y==x*x)

10
Hello Hello
Hello Hello Hello
False
True
```

Python allows multiple assignment in a single statement. You can replace the first two lines of code with `x,y = 2,4`

Note that a String can be added to another string and it can be multiplied by an integer. Try to understand the logic behind that and also try adding a String to an Integer to see what is the error message you will get. We have used the logical operator `==` for comparing two variables.

## 1.3 Operators and their Precedence

Python supports a large number of arithmetic and logical operators. They are summarized in the table. An important thing to remember is their precedence. In the expression  $2 + 3 * 4$ , is the addition done first or the multiplication? According to elementary arithmetics, the multiplication should be done first. It means that the multiplication operator has higher precedence than the addition operator. If you want the addition to be done first, enforce it by using parenthesis like  $(2 + 3) * 4$ . Whenever there is ambiguity in evaluation, use parenthesis to clarify the order of evaluation.

| Operator                                                                 | Description                             | Expression                    | Result |
|--------------------------------------------------------------------------|-----------------------------------------|-------------------------------|--------|
| <code>=, +=, -=, *=, /=, //=, %=, **=, &lt;=&lt;=, &gt;=&gt;=, @=</code> | Assignment                              | <code>x=5; x += 1</code>      | 6      |
| <code>or</code>                                                          | Boolean OR                              | <code>0 or 4</code>           | 4      |
| <code>and</code>                                                         | Boolean AND                             | <code>3 and 0</code>          | 0      |
| <code>not</code>                                                         | Boolean NOT                             | <code>not 0</code>            | True   |
| <code>in, not in, is, is not</code>                                      | Membership Tests                        | <code>3 in [2,2,3,1,2]</code> | True   |
| <code>&lt;, &lt;=, &gt;, &gt;=, !=, ==</code>                            | Comparisons                             | <code>2 &gt; 3</code>         | False  |
| <code> </code>                                                           | Bitwise OR                              | <code>1 2</code>              | 3      |
| <code>^</code>                                                           | Bitwise XOR                             | <code>1^5</code>              | 4      |
| <code>&amp;</code>                                                       | Bitwise AND                             | <code>1&amp;3</code>          | 1      |
| <code>&lt;&lt;,&gt;&gt;</code>                                           | Bitwise Shifting                        | <code>1&lt;&lt;3</code>       | 8      |
| <code>+, -</code>                                                        | Add, Subtract                           | <code>6-4</code>              | 2      |
| <code>*, //, %, @</code>                                                 | Multiply, Divide                        | <code>5%2</code>              | 1      |
| <code>+, -</code>                                                        | Positive, Negative                      | <code>-5*2</code>             | -10    |
| <code>~</code>                                                           | Bitwise NOT                             | <code>~1</code>               | -2     |
| <code>**</code>                                                          | Exponentiation                          | <code>2**3</code>             | 8      |
| <code>x[index]</code>                                                    | Function Call, Subscription, Attributes | <code>a='abcd'; a[1]</code>   | 'b'    |
| <code>()</code>                                                          | Grouping Expressions                    | <code>(5+3)*2</code>          | 16     |

## 1.4 Python Strings

So far we have come across four data types: Integer, Float, Complex and String. Out of which, String is somewhat different from the other three. It is a collection of same kind of elements, characters. The individual elements of a String can be accessed by indexing as shown in *string.py*. String is a compound, or collection, data type.

Example: string.py

```
In [4]: s = 'hello world'
        print(s[0])
        print(s[1])
        print(s[-1])
```

```
h
e
d
```

Addition and multiplication is defined for Strings, as demonstrated by *string2.py*.

Example: string2.py

```
In [5]: a = 'hello'+'world'
        print(a)
        b = 'ha'*3
        print(b)
        print(a[-1]+b[0])
```

```
helloworld
hahaha
dh
```

The last element of a and first element of b are added, resulting in the string 'dh'

### 1.4.1 Slicing

Part of a String can be extracted using the slicing operation. It can be considered as a modified form of indexing a single character. Indexing using  $s[a : b]$  extracts elements  $s[a]$  to  $s[b - 1]$ . We can skip one of the indices. If the index on the left side of the colon is skipped, slicing starts from the first element and if the index on right side is skipped, slicing ends with the last element. The reader can guess the nature of slicing operation from the output of this code, shown below.

Example: slice.py

```
In [6]: a = 'hello world'
        print(a[3:5])
        print(a[6:])
        print(a[:5])
```

```
lo
world
hello
```

Please note that specifying a right side index more than the length of the string is equivalent to skipping it. Modify `slice.py` to print the result of `a[6 : 20]` to demonstrate it.

## 1.5 Python Lists

List is an important data type of Python. It is much more flexible than String. The individual elements can be of any type, even another list. Lists are defined by enclosing the elements inside a pair of square brackets, separated by commas. The program `list1.py` defines a list and print its elements. The output is shown below.

Example: `list1.py`

```
In [7]: a = [2.3,3.5,234] # make a list
        print(a[0])
        a[1]='haha' # change an element
        print(a)
```

```
2.3
[2.3, 'haha', 234]
```

Python 2.6 and older version had a representation problem with floating point numbers. Writing 2.3 as 2.2999999999999998. Because 2.3 cannot be perfectly represented in binary, the actual computer bits store a number very slightly less than 2.3. However, since 2.3 is closer to that binary pattern than 2.2999999999999997 or 2.3000000000000001, Python 3 is smart enough to just print 2.3. It hides the "ugly" precision unless you explicitly ask to see it. To see the raw limitation in Python 3, you now have to force it to show extra decimal places:

```
In [8]: print(f"{2.3:.55f}")
```

```
2.2999999999999998223643160599749535322189331054687500000
```

Lists can be sliced in a manner similar to that if Strings. List addition and multiplication are demonstrated by the following example. We can also have another list as an element of a list. The output of this program is shown below.

Example: `list2.py`

```
In [9]: a = [1,2]
        print(a*2)
        print(a+[3,4])
        b = [10,20,a]
        print(b)

[1, 2, 1, 2]
[1, 2, 3, 4]
[10, 20, [1, 2]]
```

## 1.6 Mutable and Immutable Types

There is a major difference between String and List types. We can change the value of an element in a list, add new elements to it and remove any existing element. This is not possible with String type . Uncomment the last line of *third.py* and run it to clarify this point.

Example: *third.py*

```
In [10]: s = [3,3.5,234] # make a list
        s[2] = 'haha' # change an element
        print(s)
        x = 'myname' # string type
        #x[1] = 2 #uncomment to get Error

[3, 3.5, 'haha']
```

Another minor change in Python is **raw\_input()** function is depreciated. It is no longer available in Python3.

The List data type is very flexible, an element of a list can be another list. We will be using lists extensively in the coming chapters. Tuple is another data type similar to List, except that it is immutable. List is denoted inside square brackets, tuple is defined in a similar manner but inside parenthesis, like (3, 3.5, 234) .

## 1.7 Input from the Keyboard

Since most of the programs require some input from the user, let us introduce this feature before proceeding further. ~~There are mainly two functions used for this purpose, input() for numeric type data and raw\_input() for String type data.~~ In Python3, there is a single function *\*input()\** for this purpose. It takes inputs in the form of a string which needs to be converted explicitly by the user to the required type. A message to be displayed can be given as an argument while calling this function.

Example: *kin1.py*

```
In [11]: x = int(input('Enter an integer '))
y = int(input('Enter one more'))
print('The sum is ',x+y)
s = input('Enter a String ') #raw_input() function is no longer a
vailable
print('You entered ',s)
```

```
Enter an integer 12
Enter one more 14
The sum is 26
Enter a String hello world
You entered hello world
```

Here we have a significant change in the way Python handles multiple inputs. In Python 3, the `input()` function reads the entire line as a single string. It behaves exactly like Python 2's `raw_input()`, while Python 2's evaluating `input()` was completely removed for security reasons. To handle multiple inputs on a single line in Python 3, you must explicitly *split()* and convert that single string. And *map()* is also available to convert all the inputs to a specific data type in a single line. The `split()` method will be explained under the section 1.12.1 More on Strings and Lists towards the end of this chapter.

~~It is also possible to read more than one variable using a single `input()` statement. String type data read using `raw_input()` may be converted into integer or float type if they contain only the valid characters.~~ In order to show the effect of conversion explicitly, we multiply the variables by 2 before printing. Multiplying a String by 2 prints it twice. If the String contains any other characters than 0 ... 9, . and e , the conversion to float will give an error.

Example: kin2.py

```
In [12]: values = input('Enter x and y separated by comma ').split(',') #Reading multiple inputs separated by commas.
x,y = int(values[0]),int(values[1]) # Converting them into integer types explicitly
print('The sum is ',x+y) # We need to change them into integer type before addition.
s = input('Enter a decimal number')
a = float(s)
print(s*2) # prints string twice
print(a*2) # converted value times 2
```

```
Enter x and y separated by comma 12,14
The sum is 26
Enter a decimal number 2.4
2.42.4
4.8
```

We have learned about the basic data types of Python and how to get input data from the keyboard. This is enough to try some simple problems and algorithms to solve them.

Example: area.py

```
In [13]: pi = 3.1416
         r = float(input('Enter Radius '))
         a = pi*r**2
         print('Area = ',a)
```

```
Enter Radius 5
Area = 78.53999999999999
```

The above example calculates the area of a circle. Line three calculates  $r^2$  using the exponentiation operator `*`, and multiply it with  $\pi$  using the multiplication operator `*`.  $r^2$  is evaluated first because *has higher precedence than*, otherwise the result would be  $(\pi r^2)$ .

## 1.8 Iteration: while and for loops

If programs can only execute from the first line to the last in that order, as shown in the previous examples, it would be impossible to write any useful program. For example, we need to print the multiplication table of eight. Using our present knowledge, it would look like the following

Example: badtable.py

```
In [ ]: print(1*8)
        print(2*8)
        print(3*8)
        print(4*8)
        print(5*8)
```

Well, we are stopping here and looking for a better way to do this job. The solution is to use the while loop of Python. The logical expression in front of while is evaluated, and if it is *True*, the body of the while loop (the indented lines below the while statement) is executed. The process is repeated until the condition becomes *False*. We should have some statement inside the body of the loop that will make this condition false after few iterations. Otherwise the program will run in an infinite loop and you will have to press Control-C to terminate it. The program *table.py*, defines a variable  $x$  and assigns it an initial value of 1. Inside the while loop  $x * 8$  printed and the value of  $x$  is incremented. This process will be repeated until the value of  $x$  becomes greater than 10.

Example: table.py

```
In [ ]: x = 1
        while x <= 10:
            print(x*8)
            x = x+1
```

As per the Python syntax, the while statement ends with a colon and the code inside the while loop is indented. Indentation can be done using tab or few spaces. In this example, we have demonstrated a simple algorithm.

## 1.8.1 Python Syntax, Colon & Indentation

Python was designed to be a highly readable language. It has a relatively uncluttered visual layout, uses English keywords frequently where other languages use punctuation, and has notably fewer syntactic constructions than other popular structured languages. There are mainly two things to remember about Python syntax: indentation and colon. Python uses indentation to delimit blocks of code. Both space characters and tab characters are currently accepted as forms of indentation in Python. ~~Mixing spaces and tabs can create bugs that are hard to find, since the text editor does not show the difference.~~ There should not be any extra white spaces in the beginning of any line. The line before any indented block must end with a colon character.

Python 3 is the modern standard, it enforces a strict rule against mixing tabs and spaces and will consistently raise an error.

Python Enhancement Proposal 8 (PEP 8) is the official style guide for writing Python code, created in 2001 by Guido van Rossum (Python's creator) and Barry Warsaw. Its primary goal is to make Python code highly readable, clean, and consistent across the global developer community. The PEP 8 standard for indentation is exactly **4 spaces per indentation level**. PEP 8 strictly forbids mixing tabs and spaces for indentation, preferring spaces exclusively. You do not need to memorize all of PEP 8. Developers use automated software tools (Linters and Formatters) to check and format their code automatically.

## 1.8.2 Syntax of *for* loops

Python *for* loops are slightly different from the *for* loops of other languages. Python *for* loop iterates over a compound data type like a String, List or Tuple. During each iteration, one member of the compound data is assigned to the loop variable. The flexibility of this can be seen from the examples below.

Example: forloop.py

```
In [15]: a = 'Hello'
        for ch in a: # ch is the loop variable
            print(ch)
        b = ['haha',3.4,2345,3+5j]
        for item in b:
            print(item)
```

```
H
e
l
l
o
haha
3.4
2345
(3+5j)
```

For constructing for loops that executes a fixed number of times, we can create a list using the *range()* function and run the for loop over that.

Example: forloop2.py

```
In [16]: mylist = range(5)
         print(mylist)
         for item in mylist:
             print(item)
```

```
range(0, 5)
0
1
2
3
4
```

The range function in the above example generates the list `[0, 1, 2, 3, 4]` and the for loop walks through it printing each member. It is possible to specify the starting point and increment as arguments in the form *range(start, end+1, step)*. The following example prints the table of 5 using this feature.

Example: forloop3.py

```
In [17]: a = [2,5,3,4,12]
         size = len(a)
         for k in range(size):
             a[k] = 0
         print(a)
```

```
[0, 0, 0, 0, 0]
```

Python's built-in *range()* function does not allow float types. If you pass a float value as an argument to *range()*, Python will immediately throw a *TypeError: 'float' object cannot be interpreted as an integer*. The built-in function strictly requires whole integers for its start, stop, and step arguments. If you need to generate a sequence of floating-point numbers, the *arange()* or *linspace()* functions from the NumPy Library are the standard industry solutions. We will discuss them in Module 2. Or you could find a way around it.

## 1.9 Conditional Execution: if, elif and else

In some cases, we may need to execute some section of the code only if certain conditions are true. Python implements this feature using the *if*, *elif* and *else* keywords, as shown in the next example. The indentation levels of *if* and the corresponding *elif* and *else* must be kept the same.

Example: compare.py

```
In [18]: x = input('Enter a string')
        if x == 'hello':
            print('You typed ',x)
```

```
Enter a stringhello
You typed  hello
```

Example: big.py

```
In [19]: x = int(input('Enter a number'))
        if x > 10:
            print('Bigger Number')
        elif x < 10:
            print('Smaller Number')
        else:
            print('Same Number')
```

```
Enter a number8
Smaller Number
```

The statement  $x > 10$  and  $x < 15$  can be expressed in a short form, like  $10 < x < 15$ . The next example uses *while* and *if* keywords in the same program. Note the level of indentation when the *if* statement comes inside the *while* loop. Remember that, then *if* statement must be aligned with the corresponding *elif* and *else*.

Example: big2.py

```
In [20]: x = 1
        while x < 11:
            if x < 5:
                print('Small ',x)
            else:
                print('Big ',x)
            x = x+1
        print('Done')
```

```
Small  1
Small  2
Small  3
Small  4
Big    5
Big    6
Big    7
Big    8
Big    9
Big    10
Done
```

## 1.10 Modify loops: break and continue

We can use the *break* statement to terminate a loop, if some condition is met. The *continue* statement is used to skip the rest of the block and go to the beginning again. Both are demonstrated in the program `big3.py` shown below.

Example: `big3.py`

```
In [21]: x = 1
while x <= 10:
    if x < 3:
        print('Skipping work',x)
        x = x+1
        continue
    print(x)
    if x==4 :
        print('Enough of work')
        break
    x = x+1
print('Done')
```

```
Skipping work 1
Skipping work 2
3
4
Enough of work
Done
```

Now let us write a program to find out the largest positive number entered by the user. The algorithm works in the following manner. To start with, we assume that the largest number is zero. After reading a number, the program checks whether it is bigger than the current value of the largest number. If so the value of the largest number is replaced with the current number. The program terminates when the user enters zero. Modify `max.py` to work with negative numbers also.

Example: `max.py`

```
In [22]: max = 0
while True: #infinite loop
    x = float(input('Enter a number '))
    if x > max:
        max = x
    if x == 0:
        print(max)
        break
```

```
Enter a number 13
Enter a number 14
Enter a number 10
Enter a number 0
14.0
```

## \*Line joining

Python interpreter processes the code line by line. A program may have a long line of code that may not physically fit in the width of the text editor. In such cases, we can split a logical line of code into more than one physical lines, using backslash characters (`\`), in other words multiple physical lines are joined to form a logical line before interpreting it

Python manages long lines of code using two distinct methods: Implicit Line Joining (highly recommended) and Explicit Line Joining (discouraged). This section explains explicit line joining which was the only way before Python3.

```
In [ ]: month, year = 2,1996
        if 1900 < year < 2100 \
            and 1 <= month <= 12:
            print('Yes')
```

```
In [ ]: longname = 'I am so long an will\
                  not fit in a single line'
        print(longname)
```

The Best Practice on Python3 are Implicit Line Joining which uses paranthesis to group those mutiple lines. And long strings spanning multiple lines are automatically appended.

```
In [23]: month, year = 2,1996
        if (1900 < year < 2100
            and 1 <= month <= 12):
            print('Yes')

        longname = (
            'I am so long an will '
            'not fit in a single line'
        )
        print(longname)

Yes
I am so long an will not fit in a single line
```

## Exercises 1

1. Modify the expression `print 5+3*2` to get a result of 16.

```
In [ ]: print((5+3)*2) #print (5+3)*2 won't work in Python3
```

**2. What will be the output of print(type(4.5))**

```
In [24]: print(type(4.5))  
  
<class 'float'>
```

**3. Print all even numbers upto 30, suffixed by a \* if the number is a multiple of 6.(hint: use % operator)**

```
In [ ]: for i in range(1,31):  
        if i%6==0 :  
            print('*',i)  
        elif i%2==0:  
            print(i)
```

**4. Write Python code to remove the last two characters of 'I am a long string' by slicing, without counting the characters. (hint: use negative indexing)**

```
In [25]: s = 'I am a long string'  
s = s[:-2]  
print(s)  
  
I am a long stri
```

**5. s = '012345'. a) Slice it to remove last two elements b) remove first two elements.**

```
In [26]: s = '012345'  
print(s[:-2])  
print(s[2:])  
  
0123  
2345
```

**6. a = [1,2,3,4,5]. Use Slicing and multiplication to generate [2,3,4,2,3,4] from it.**

```
In [27]: a = [1,2,3,4,5]  
print(a[1:-1]*2)  
  
[2, 3, 4, 2, 3, 4]
```

**7. Compare the results of 5/2, 5.0/2 and 2.0/3**

You don't have to use a mixed expression to force implicit int to float type conversion when the operation is division. And decides whether to convert depending on the nature of exponent when it comes to exponentiation.

```
In [28]: print(5/2) #Python3 implicitly converts int to float before division
print(5.0/2)
print(2.0/3)

2.5
2.5
0.6666666666666666
```

#### 8. Print the following pattern using a while loop

```

+
++
+++
++++
```

```
In [ ]: s = '+'
        for i in range(1,5):
            print(s*i)
```

#### 9. Write a program to read inputs 8A,10C etc. and print the integer and alphabet parts separately.

Longer alphabet part demand use of *isnumeric()*,*isalpha()* functions which we will discuss in 1.12 More on Strings and Lists.

```
In [29]: Class = input('Enter class name ')
std = Class[:-1]
div = Class[-1]
print('Standard ',std,' Division ',div)

Enter class name 7B
Standard 7 Division B
```

#### 10. Write code to print a number in the binary format (for example 5 will be printed as 101)

The number is divided by 2 and the remainders are added at  $10^j$ th place.

```
In [30]: n = int(input('Number : '))
        binary,j = 0,0
        while n > 0 :
            binary += (n%2)*10**j
            j += 1
            n //= 2
        print(binary)
```

```
Number : 8121
1111110111001
```

**11. Write code to print all perfect cubes upto 2000.**

```
In [31]: j = 1
        while j**3 < 2000:
            print(j**3)
            j += 1
```

```
1
8
27
64
125
216
343
512
729
1000
1331
1728
```

**12. Write a Python program to print the multiplication table of 5.**

```
In [32]: for j in range(1,11):
        print(j,' x ',5,' = ',j*5)
```

```
1 x 5 = 5
2 x 5 = 10
3 x 5 = 15
4 x 5 = 20
5 x 5 = 25
6 x 5 = 30
7 x 5 = 35
8 x 5 = 40
9 x 5 = 45
10 x 5 = 50
```

13. Write a program to find the volume of a box with sides 3,4 and 5 inches in  $cm^3$  (1 inch = 2.54 cm)

```
In [33]: scale = 2.54 # 1 inch = 2.54 cm
a,b,c = 3*scale, 4*scale,5*scale
volume = a*b*c
print('Volume of Box = ',volume,'cubic cm')

Volume of Box = 983.22384 cubic cm
```

14. Write a program to find the percentage of volume occupied by a sphere of diameter  $r$  fitted in a cube of side  $r$ . Read  $r$  from the keyboard.

```
In [34]: d = float(input('Diameter :'))
s = 4/3*3.1416*(d/2)**3
c = d**3
ratio = (c-s)*100/c
print('Percentage of Volume occupied by Sphere in Cube = ',ratio, ' %')

Diameter :3
Percentage of Volume occupied by Sphere in Cube = 47.640000000000001 %
```

15. Write a Python program to calculate the area of a circle.

```
In [35]: r = float(input('Radius :'))
area = 3.1416*r**2
print('Area of Circle = ',area)

Radius :8
Area of Circle = 201.0624
```

16. Write a program to divide an integer by another without using the / operator. (hint: use - operator)

```
In [36]: dividend = int(input('Dividend :'))
divisor = int(input('Divisor :'))
quotient = 0
while dividend > divisor:
    quotient += 1
    dividend -= divisor
print('Quotient =',quotient,' Remainder =',dividend)

Dividend :181
Divisor :29
Quotient = 6 Remainder = 7
```

**17. Count the number of times the character 'a' appears in a string read from the keyboard. Keep on prompting for the string until there is no 'a' in the input.**

```
In [37]: s = input('String :')
        while s.count('a') > 0:
            s = input('String :')
```

```
String :I am long string
String :hello world
```

**18. Create an integer division machine that will ask the user for two numbers then divide and give the result. The program should give the result in two parts : the whole number result and the remainder. Example: If a user enters 11/4, the computer should give the result 2 and remainder 3.**

```
In [38]: values = input('Division expression :').split('/')
        dividend,divisor = int(values[0]),int(values[1])
        print('Quotient =',dividend//divisor,' Remainder =',dividend%divisor)
```

```
Division expression :114/23
Quotient = 4  Remainder = 7
```

**19. Modify the previous program to avoid division by zero error. (Out of Syllabus)**

**20. Create an adding machine that will keep on asking the user for numbers, add them together and show the total after each step. Terminate when user enters a zero.**

```
In [39]: n = float(input('Number :'))
        sum = n
        while n != 0:
            n = float(input('Number :'))
            sum += n
            print('Sum =',sum)
```

```
Number :17
Number :19
Sum = 36.0
Number :21
Sum = 57.0
Number :0
Sum = 57.0
```

**21. Modify the adding machine to raw\_input() and check for errors like user entered invalid characters. (Out of Syllabus)**

**22. Create a script that will convert Celsius to Fahrenheit. The program should ask the user to enter the temperature in Celsius and should print out the temperature in Fahrenheit, using**

$$f = \frac{9}{5}c + 32.$$

```
In [40]: c = float(input('Temperature (Celsius):'))
         f = 9*c/5+32
         print('Temperature (Fahrenheit) =',f)
```

```
Temperature (Celsius):36.3
Temperature (Fahrenheit) = 97.34
```

**23. Write a program to convert Fahrenheit to Celsius.**

```
In [41]: f = float(input('Temperature (Fahrenheit):'))
         c = (f-32)*5/9
         print('Temperature (Celsius) =',c)
```

```
Temperature (Fahrenheit):99.3
Temperature (Celsius) = 37.388888888888886
```

**24. Create a script that uses a variable and will write 20 times 'I will not talk in class.' Make each sentence on a separate line.**

```
In [ ]: s = 'I will not talk in class.'
         for i in range(20):
             print(s)
```

**25. Define 2+5j and 2-5j as complex numbers, and find their product. Verify the result by defining the real and imaginary parts separately and using the multiplication formula.**

```
In [42]: z1 = 2+5j
         z2 = 2-5j
         product1 = z1*z2
         product2r = z1.real*z2.real-z1.imag*z2.imag
         product2i = z1.real*z2.imag+z1.imag*z2.real
         if product1.real == product2r and product1.imag == product2i :
             print('Both products verified to be', product1)
         else:
             print('Verification Failed')
```

```
Both products verified to be (29+0j)
```

**26. Write the multiplication table of 23 using while loop.**

```
In [ ]: n = 1
         while n < 11:
             print(n,' x ',12,' = ',n*12)
             n += 1
```

27. Write the multiplication table of a number, from the user, using for loop.

```
In [ ]: n = int(input('Number :'))
        for i in range(1,11):
            print(i, ' x ', n, ' = ', i*n)
```

28. Print the powers of 2 up to 1024 using a for loop. (only two lines of code)

```
In [43]: for i in range(1,11):
          print(2**i)
```

```
2
4
8
16
32
64
128
256
512
1024
```

29. Define the list a = [123,12.4,'haha',3.4] a) Print all members using a for loop b) Print the float type members (use type() function) c) Insert a member after 12.4 d) Append more members

```
In [44]: a = [123,12.4,'haha',3.4]
        for item in a:
            print(item)
        for item in a:
            if type(item) == type(2.3): #filtering items of type float.
                print(item)
        a.insert(2,2+5j)
        a.append({1,2})
        a.append((1,2))
        a.append([1,2])
        print(a)
```

```
123
12.4
haha
3.4
12.4
3.4
[123, 12.4, (2+5j), 'haha', 3.4, {1, 2}, (1, 2), [1, 2]]
```

30. Make a list containing 10 members using a for loop.

```
In [45]: a = []
         for i in range(10):
             a.append(i**2)
         print(a)
```

```
[0, 1, 4, 9, 16, 25, 36, 49, 64, 81]
```

### 31. Generate multiplication table of 5 with two lines of Python Code

```
In [ ]: for j in range(1,11):
         print(j, ' x ', 5, ' = ', j*5)
```

### 32. Write a program to find the sum of five numbers read from the keyboard.

```
In [46]: values = list(map(int, input('Enter five numbers:').split(',')))
         sum = 0
         for i in range(5):
             sum += values[i]
         print('Sum of (first) Five Numbers =', sum)
```

```
Enter five numbers:1,2,3,4,5,6
Sum of (first) Five Numbers = 15
```

### 33. Write a program to read numbers from the keyboard until their sum exceeds 200. Modify the program to ignore numbers greater than 99.

```
In [47]: sum = 0
         while sum < 200:
             n = float(input('Number :'))
             if n > 99: #Ignoring Numbers greater than 99
                 continue
             sum += n
             print('Sum =', sum)
```

```
Number :97
Sum = 97.0
Number :23
Sum = 120.0
Number :103
Number :83
Sum = 203.0
```

### 34. Write a Python function to calculate the GCD of two numbers.

Euclidean Algorithm :  $\text{gcd}(x,y) = \text{gcd}(y,x\%y)$  and  $\text{gcd}(x,0) = x$

For example,  $\text{gcd}(10,44) = \text{gcd}(44,10) = \text{gcd}(10,4) = \text{gcd}(4,2) = \text{gcd}(2,0) = 2$

```
In [48]: def gcd(x,y):
          if y == 0:
              print(x)
          else :
              gcd(y,x%y)

gcd(3287634,3722718)

6
```

**35. Write a Python program to find annual compound interest. Get P,N,and R from user.**

```
In [49]: P = float(input('Principal Amount :'))
          N = int(input('Number of Years :'))
          R = float(input('Rate of Interest :'))
          CI = ((1+R/100)**N-1)*P #  $P(r^{*}N)-P$  where  $r = R/100$ 
          print('Compound Interest =',CI)

Principal Amount :12000
Number of Years :3
Rate of Interest :5.5
Compound Interest = 2090.8964999999976
```

---

## 1.11 Functions

```
In [50]: def sum(a,b): # a trivial function
          return a+b

print(sum(3,4))

7
```

```
In [51]: def factorial(n): #a recursive function
          if n == 0:
              return 1
          else :
              return n*factorial(n-1)

print(factorial(10))

3628800
```

```
In [52]: def fib(n): # print fibonacci series upto n
          a,b = 0,1
          while b < n:
              print(b)
              a,b = b,a+b

          fib(30) # changed slightly to avoid the return value None getting
               printed.

1
1
2
3
5
8
13
21
```

### 1.11.1 Scope of variables

```
In [53]: def change(x):
          counter = x

          counter = 10
          change(5)
          print(counter)
```

10

```
In [54]: def change(x):
          global counter #use the global variable
          counter = x

          counter = 10
          change(5)
          print(counter)
```

5

### 1.11.2 Optional and Named Arguments

```
In [55]: def power(mant=10.0,exp=2.0):
          return mant**exp

          print(power(5,3))
          print(power(4))
          print(power(exp=3))
```

125  
16.0  
1000.0

## 1.12 More on Strings and Lists

```
In [56]: s = 'hello world'
         print(len(s))
         print(s.upper())
         #help(str) # press q to exit help

11
HELLO WORLD
```

### 1.12.1 split() and join()

```
In [57]: s = 'I am a long string'
         print(s.split())
         a = 'abc.abc.abc'
         aa = a.split('.')
         print(aa)
         mm = '+'.join(aa)
         print(mm)

['I', 'am', 'a', 'long', 'string']
['abc', 'abc', 'abc']
abc+abc+abc
```

### 1.12.2 Manipulating Lists

```
In [58]: a = [1,2,3,4]
         print(a)
         b = a # b refers to a
         print(a == b) # True
         b[0] = 5 # modifies a[0]
         print(a)
         import copy
         c = copy.copy(a)
         c[1] = 100
         print(a is c) # False
         print(a,c)

[1, 2, 3, 4]
True
[5, 2, 3, 4]
False
[5, 2, 3, 4] [5, 100, 3, 4]
```

## 1.13 Python Modules and Packages

### 1.13.1 Different ways to import

```
In [59]: import math
print(math.sin(0.5)) # module_name.method_name

0.479425538604203
```

```
In [60]: import math as m # An alias for math
print(m.sin(0.5)) # refer math by new name

0.479425538604203
```

```
In [61]: from math import sin # sin is imported as local
print(sin(0.5))

0.479425538604203
```

```
In [62]: from math import * # importing everything from math
print(sin(0.5))

0.479425538604203
```

```
In [63]: from numpy import *
x = [0.1,0.2,0.3]
print(sin(x)) #numpy.sin can handle lists
from math import * #math.sin becomes effective
#print(sin(x)) # Gives Error

[0.09983342 0.19866933 0.29552021]
```

## 1.13.2 Packages

```
In [64]: import numpy #import numpy imports numpy.random
print(numpy.random.normal())
import scipy.special # import scipy does not import scipy.special
print(scipy.special.j0(.1))

-0.07924730410148008
0.99750156206604
```

## Exercises 2

1. Generate multiplication table of eight and write it to a file.(Out of Syllabus)
2. Make a list and write it to a file using pickle module. (Out of Syllabus)
3. Write a Python program to open a file and write 'hello world' to it. (Out of Syllabus)

4. Write a Python program to open a text file and read all lines from it. (Out of Syllabus)

5. Write a program to generate the multiplication table of a number from the user. The output should be formatted as shown below.(Out of Syllabus - f Strings)

1 x 5 = 5

2 x 5 = 10

6. Define the list [1,2,3,4,5,6] using the range function. Write code to insert a 10 after 2, delete 4, add 0 at the end and sort it in the ascending order.

```
In [65]: a = list(range(1,7))
          a.insert(2,10)
          a.remove(4)
          a.append(0)
          a.sort()
          print(a)

[0, 1, 2, 3, 5, 6, 10]
```

7. Write Python code to generate the sequence of numbers 25 20 15 10 5 using range function. Delete 15 from the result and sort it. Print it using a for loop.

```
In [66]: a = list(range(25,0,-5))
          a.remove(15)
          a.sort()
          for item in a:
              print(item)

5
10
20
25
```

8. Define a string s= 'mary had a little lamb'. a) Print it in reverse order b) Split it using space character as separator.

```
In [67]: s = 'mary had a little lamb'
          r = ''
          for c in s:
              r = c+r
          print(r)
          print(s.split())

bmal elttil a dah yram
['mary', 'had', 'a', 'little', 'lamb']
```

9. Join the elements of the list ['I','am','in','pieces'] using + character. Do the same using a for loop.

```
In [68]: l = ['I','am','in','pieces']
print('+'.join(l))
s = ''
for ch in l:
    s += ch
    s += '+'
s = s[:-1]
print(s)
```

```
I+am+in+pieces
I+am+in+pieces
```

10. Create a window with five buttons. Make each button a different colour. Each button should have some text on it. (Out of Syllabus)

11. Create a program that will put words in alphabetical order. The program should allow the use to enter as many words as he wants to.(Out of Syllabus)

12. Create a program that will check a sentence to see if it is a palindrome. A palindrome is a sentence that reads the same backwards and forwards. ('malayalam')

```
In [70]: s = input('Check Palindrome:')
l = len(s)
for k in range(l):
    if s[k] != s[l-k-1]:
        print('It is not a palindrome.')
        break
if k == l-1:
    print('It is a palindrome.')
```

```
Check Palindrome:malayalam
It is a palindrome.
```

13. A text file contains two columns of numbers. Write a program to read them and print the sum of numbers in each row. (Out of Syllabus)

14. Read a string from the keyboard. Multiply it by an integer to make its length more than 50. How do you find out the smallest number that does the job.

```
In [71]: s = input('Enter a String :')
l = len(s)
if 50%l == 0:
    n = 50//l
else:
    n = 50//l+1
print(n,s*n)
```

```
Enter a String :HelloWorld
5 HelloWorldHelloWorldHelloWorldHelloWorldHelloWorld
```

**15. Write a program to find the length of the hypotenuse of a right triangle from the length of other two sides, get the input from the user.**

```
In [72]: a = float(input('Baselenght of Triangle:'))
b = float(input('Height of Triangle:'))
h = (a**2+b**2)**0.5
print('Length of Hypotenuse =',h)
```

```
Baselenght of Triangle:6
Height of Triangle:8
Length of Hypotenuse = 10.0
```

**16. Write a program displaying 2 labels and 2 buttons. It should print two different messages when clicked on the two buttons. (Out of Syllabus)**

**17. Write a program with a Canvas and a circle drawn on it. (Out of Syllabus)**

**18. Write a program using for loop to reverse a string.**

```
In [73]: s = input('Enter a String:')
r = ''
for c in s:
    r = c+r
print(r)
```

```
Enter a String:Hello World
dlroW olleH
```

**19. Write a Python function to calculate the GCD of two numbers.**

```
In [74]: def gcd(x,y):
        if y == 0:
            print(x)
        else:
            gcd(y,x%y)
gcd(60,42)
```

20. Write a program to print the values of sine function from 0 to  $2\pi$  with 0.1 increments. Find the mean value of them.

```
In [75]: from math import sin
r = range(0,2*31416,1000)
l = []
for j in r:
    l.append(sin(j/10000))
print(sum(l)/len(l))

-0.00011102368734680821
```

21. Generate N random numbers using random.random() and find out how many are below 0.5. Repeat the same for different values of N to draw some conclusion.

```
In [76]: from random import random
n = int(input('Number of Random Numbers :'))
count = 0
for j in range(n):
    r = random()
    if r < 0.5:
        count += 1
print(count)

Number of Random Numbers :39
19
```

22. Use the equation  $x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$  to find the roots of  $3x^2 + 6x + 12 = 0$ .

```
In [77]: a = 3
b = 6
c = 12
r1 = (-b+(b**2-4*a*c))/(2*a)
r2 = (-b-(b**2-4*a*c))/(2*a)
print('Roots are ',r1,r2)

Roots are  -19.0 17.0
```

23. Write a program to calculate the distance between points (x1,y1) and (x2,y2) in the Cartesian plane. Get the coordinates from the user.

```
In [78]: x = tuple(map(int,input('Enter Coordinates of X :').split(',')))
#map converts each string into int type
y = tuple(map(int,input('Enter Coordinates of Y :').split(',')))
distance = ((x[0]-y[0])**2+(x[1]-y[1])**2)**0.5
print('Distance = ',distance)

Enter Coordinates of X :2,3
Enter Coordinates of Y :-3,5
Distance =  5.385164807134504
```

24. Write a program to evaluate  $y = \sqrt{2.3a} + a^2 + 34.5$  for  $a = 1, 2$  and  $3$ .

```
In [79]: f = lambda x : (2.3*x)**0.5+x**2+34.5 #lambda functions are one-liners
print(f(1),f(2),f(3))
```

```
37.01657508881031 40.644761058952724 46.12678510731274
```

25. Print Fibonacci numbers upto 100, without using multiple assignment statement.

```
In [80]: a = 0
b = 1
while b < 100 :
    print(b)
    temp = a
    a = b
    b = temp + b
```

```
1
1
2
3
5
8
13
21
34
55
89
```

26. Draw a chess board pattern using turtle graphics. (Out of Syllabus)

27. Find the syntax error in the following code and correct it.

```
In [ ]: x = 1
while x <= 10:
print x*5
```